

Komplett Lathund: Test kap. 5–8 (Teori, Syntax & Loopar)

Lathund Python: Felhantering, Datastrukturer, Moduler och Funktionell Programmering

Kapitel 5: Felhantering och Felsökning

Naiv felhantering: Att enbart returnera en sträng eller `int` vid fel. Detta indikerar att ett fel uppstått, men ger ingen detaljerad information om *vad* som gick fel.

Try / Except: Rekommenderat sätt att hantera undantag (exceptions). Målet är att ha så lite kod som möjligt inuti try-blocket för att isolera felkällan.

Vanliga Särfall (Exceptions):

- `ValueError`: Rätt datatyp, men ogiltigt värde. T.ex. `int("hej")`.
- `FileNotFoundError`: Filen som försöker öppnas existerar inte.
- `IOError` (även `OSError`): Fel vid in/utmatning, t.ex. filen är låst.
- `ZeroDivisionError`: Division med noll.
- `IndexError`: Försök att komma åt ett index utanför en listas gränser.
- `KeyError`: Försök att använda en nyckel som inte finns i en dictionary.
- `TypeError`: Operation applicerad på fel datatyp (mer generellt än `ValueError`). T.ex. `"hej" + 5`.

Hantering av fel:

```
while True:
    try:
        x = int(input("Ange ett tal: "))
        break # Bryter loopen om inmatningen lyckas
    except ValueError as fel: # Sparar felet för ev. utskrift
        print(f"Ett fel uppstod: {fel}. Försök igen.")
        continue # Startar om loopen
```

Skapa egna fel: Använd `raise` för att manuellt kasta ett undantag.

```
raise Exception("Mitt anpassade felmeddelande")
```

Designval vid felhantering:

- **Applikationsprogrammering (Slutanvändare):** Fånga fel (`try/except`) för att förhindra krascher och hålla programmet igång för användaren.
- **Stödprogrammering (Bibliotek/Moduler):** Låt koden krascha direkt (fail-fast", `raise`) så att utvecklaren upptäcker felet.
- **If vs Try/Except:** Använd `if` för förväntad logik och `try/except` för oförutsägbara fel. `try/except` kan dra mer prestanda när undantag faktiskt kastas.
- Hantera felen, ignorera dem inte tyst. Exempel: Vid `FileNotFoundError`, ge användaren valet att skapa filen eller avsluta via `quit()`.

Comprehensions (Omfattningar): Ett kompakt sätt att skapa samlingar, inspirerat av matematik.

List Comprehensions syntax: `[uttryck for item in iterable if villkor]`

```
# Skapar en lista med kvadrater av jämna tal (0 till 9)
kvadrater = [x**2 for x in range(10) if x % 2 == 0]
```

Tips: Använd `zip(lista1, lista2)` för att iterera över två listor samtidigt i en comprehension.

Dict Comprehensions syntax: `{nyckel: värde for item in iterable if villkor}`

```
# Skapar en dictionary med jämna tal som nycklar och deras kvadrater som värden
kvadrat_dict = {x: x**2 for x in range(10) if x % 2 == 0}
```

Felsökningstekniker: Leta efter *orsaken*, inte bara *symptomet*.

- **Spårutskrifter (Print-debugging):** Använd f-strängar `f'{x=}'` för att skriva ut både variabelns namn och värde (t.ex. `x=5`). Ta bort dessa innan koden är färdig.
- **Logging:** Mer professionellt alternativ till `print()`. Kan slås av/på och dirigeras till filer.

```
import logging
# Sätter loggnivå till DEBUG (skriver ut allting) och mål till en fil
logging.basicConfig(filename='derivative.log', level=logging.DEBUG)

def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:
        new_coefficient = term * degree
        # Skriver information till loggfilen
        logging.debug(f'{degree=}, {term=}, {new_coefficient=}')
        result.append(new_coefficient)
        degree += 1
    return result
```

Loggnivåer: Om nivån sätts högre än DEBUG (t.ex. WARNING eller ERROR), ignoreras `logging.debug()`.

- **Debugger (i IDE som VS Code/Spyder):** Låter dig köra koden rad för rad.
 - **Brytpunkt (Breakpoint):** Pausar exekveringen på en specifik rad.
 - **Step Over:** Kör nuvarande rad och går till nästa.
 - **Step Into:** Går *in* i funktionen som anropas på nuvarande rad.
 - **Continue:** Kör koden tills nästa brytpunkt träffas.
- **Kontrollpunkter (assert):** Avbryter programmet om ett villkor är falskt.

```
assert x > 0, "x måste vara positivt" # Kastar AssertionError om x <= 0
```

- **Övriga strategier:** skriv om kort kod för att undvika stavfel, dubbelkolla format på indata är rätt tecken även om de är lika eller åäö (är specifikt för varje dator), arbeta med små testdata, testa specialfall som används

Kapitel 6: Sekvenstyper och Generatorer

Sekvenstyper: Gemensamma operationer: `+`, `*`, `in`, `not in`, `len()`, `min()`, `max()`, `[:]`. Sekvensoperationer returnerar objekt av samma typ. En sekvenstyp är en samling av element som kan indexeras och itereras över medan en generator är en iterator som genererar ett element i taget.

- **Listor []:** Muterbara (föränderliga) sekvenstyp.
- **Dictionaries {}:** Muterbara sekvenstyp.
- **Strängar :** Immutabla (oföränderliga) sekvenstyp. Har många inbyggda metoder (t.ex. `lower()`, `split()`).
- **Tuples ():** Immutabla sekvenstyp. Används för samlingar som inte ska ändras. Kan användas som nycklar i dictionaries (förutsatt att de inte innehåller muterbara objekt som en lista).
- **Range range(start, stop, step):** Immutabel generator. Stödjer inte `+` eller `*`.
- **Yield:** Används i generatorfunktioner för att returnera ett värde i taget. Jämfört med `return` så sparas tillståndet i funktionen och kan återupptas vid nästa anrop.

Generatorer & Evaluering:

- **Strikt evaluering:** Alla värden beräknas och lagras i minnet direkt (t.ex. listor, strängar).
- **Lat evaluering (Generatorer):** Itererbara objekt som genererar ett värde i taget vid behov (`yield` istället för `return`). Mycket minneseffektivt för stora datamängder (t.ex. `range()`).

Skapa Generatorer:

```
# Generatorfunktion med yield
def min_generator():
    yield 1
    yield 2

# Generatoruttryck (syntax liknande list comprehension fast med parenteser)
gen = (x**2 for x in range(10) if x % 2 == 0)
```

Kapitel 7: Moduler och Bra Kod

Modulsystemet: Varje .py-fil är en modul.

```
import math # Hela modulen. Användning: math.sqrt()
from math import sqrt # Specifik funktion. Användning: sqrt()
import numpy as np # Import med alias. Användning: np.array()
# from math import * # Ofta dålig praxis (kan skapa namnkonflikter)
```

Paket: Mapper som grupperar flera relaterade moduler.

__main__ blocket: Används för kod som endast ska köras om filen exekveras direkt (inte när den importeras).

```
def min_funktion():
    pass

if __name__ == "__main__":
    # Denna kod körs inte om filen importeras som modul
    print("Körs som huvudprogram")
```

Att skriva egna moduler: Först Skriv kod i en pythonfil, sen Spara filen med .py, sist Importera modulen i en annan fil med import modulnamn. OBS python letar först i samma mapp, så ha gärna modulen där.

Vanliga Bibliotek: Är samling av moduler som kan importeras: math, sys, os, argparse, itertools, functools, datetime, pickle (serialisering), csv, json, matplotlib, numpy, scipy.

JSON och Serialisering: Löser problemet med att spara komplexa datastrukturer (nästlade listor/dict) till fil i ett portabelt textformat.

```
import json
data = {"namn": "Ada", "ålder": 30}
json_string = json.dumps(data) # Konverterar dict till JSON-sträng
```

Kodkvalitet & PEP-8: Python stilguide. Man kan även kolla att sin kod är rätt med pythochecker.com eller IDE med stöd samt pylint i terminalfönstret.

- **Kommentarer:** Ska förklara *varför*, inte *hur*. Använd docstrings "... " för funktioner/klasser.
- **Funktioner:** Ska vara korta (max en halv skärm), göra *en* sak, och oftast ta parametrar och returnera värden (skriv inte ut i funktionen om det inte är dess uttalade syfte). Undvik att returnera magiska siffror (siffror utan betydelse för användaren), returnera hellre flera värden som en tuple än komplexa blandade strängar.
- **Struktur:** Skapa top-down. Abstrahera bort detaljer i modulär kod. En instruktion per rad. Uppdatera dessa vanor när någon del av koden uppdateras. Om indentering i if satser bli djupa, skapa hjälpfunktioner för fallen.
- **Variabler:** Undvik globala variabler (globala konstanter är OK). Ge beskrivande namn, undvik magiskanummer direkt i koden.
- **Tester:** Det ska vara lätt att skapa tester mha modulär kod.
- **Undantag:** Funktioner som main behöver inte vara informativa.
- **Dålig praxis:** Copy-paste av kod (skriv om istället).

Kapitel 8: Funktionell Programmering

Funktionell programmering: Paradigmen (sätt att programmera”) där funktioner kan anropa, sätta ihop och modifiera andra funktioner. Detta kontrasteras mot imperativa språk då man skriver steg-för-steg instruktioner, dvs den programmering man lär sig först.

First Class Citizen (Första klassens medborgare): Innebär att funktioner behandlas som vilken annan variabel som helst: de kan skickas som argument, returneras och tilldelas.

Rekursion: När en funktion anropar sig själv. Måste ha ett **basfall** för att sluta anropa sig själv. Riskerar dock att bli långsamma och kan effektiviseras genom att skapa flera delsteg i argumentet och funktionen för att täcka flera fall samtidigt.

- **Användningsområden:** Matte (Fibonacci, GCD), loopa över tal/listor. Vid listor används ofta en tom lista [] som basfall, och man plockar bort element i varje anrop.
- **Quicksort:** Ett klassiskt rekursivt exempel. Välj pivot, dela i listor med större/mindre element, anropa rekursivt, sätt ihop.
- **Minneshantering (Stacken):** Vid varje anrop sparas parametrar och returadresser på anrops-stacken. Vid djupt nästlade anrop kan detta dra mycket minne.
- **Svansrekursion (Tail Recursion):** Om det rekursiva anropet är det absolut sista som sker, kan kompilatorer (vissa, men inte Python standard) optimera bort stack-användningen till en vanlig loop.

Anonyma funktioner (Lambda): Små engångsfunktioner. *Syntax:* `lambda argument: uttryck`

```
kvadrat = lambda x: x**2
```

Högre ordningens funktioner: Funktioner som tar andra funktioner som argument.

- **map:** Applicerar en funktion på alla element i en iterabel.

```
lista = [1, 2, 3]
# Applicerar lambda-funktionen på varje element
kvadrater = list(map(lambda x: x**2, lista))
```

- **filter:** Tar ett predikat (en funktion som returnerar True/False) och returnerar de element som är True.

```
# Filtrerar ut de jämna talen
jämna = list(filter(lambda x: x % 2 == 0, lista))
```

- **Egna ordningens högre funktioner:** Man kan skapa egna högre ordningens funktioner med `def` som tar funktioner som argument.

```
def apply_function(func, data):
    return [func(x) for x in data]
# Exempelanvändning
resultat = apply_function(lambda x: x**2, [1, 2, 3])
```

Questions

1. Placering av try-except i loopar (Kapitel 5, Uppgift 2)

- **Viktigt att veta (Placering och Flöde):**
 - **try-except Utanför/Runt loopen:** Om `try`-blocket omsluter hela `for`-loopen kommer programmet att avbryta hela loopen så fort ett särfall (exception) uppstår på någon rad. Kontrollflödet hoppar direkt ut ur loopen till `except`-blocket och återvänder inte till efterföljande rader.
 - **try-except Inuti loopen:** Om `try-except` placeras inuti `for`-loopen hanteras felet enbart för den aktuella iterationen. När `except`-blocket har exekverats fortsätter loopen automatiskt till nästa element/rad i filen.

- **Hur man löser problemet:**

- Placera try-blocket inuti loopen runt den specifika beräkningen och typomvandlingen.
- Fånga de specifika felen (t.ex. `ZeroDivisionError` och `ValueError`) i separata `except`-block så att giltiga rader fortfarande kan beräknas och läggas till i resultatlistan.

- **Kodexempel:**

```
def divide_by_elems(filename, x):
    quotients = []
    with open(filename, 'r') as h:
        for n in h:
            # try-except placeras INUTI loopen
            try:
                frac = x / int(n)
                quotients.append(frac)
            except ZeroDivisionError:
                print("divide_by_elems: Division med noll ignoreras.")
            except ValueError:
                print("divide_by_elems: Kan inte konvertera raden till ett heltal.")
    return quotients
```

2. Nästlad Listomfattning med Indexering (Kapitel 5, Uppgift 7)

- **Viktigt att veta (Hur två for-delar samverkar):**

- En nästlad listomfattning (list comprehension) läses från vänster till höger på samma sätt som nästlade for-loopar.
- Den första `for i in ...`-delen utgör den yttre loopen, och den andra `for j in ...`-delen utgör den inre loopen.
- För varje värde på index i köra hela j -loopen igenom alla sina index.

- **Varför vi kollar $i \neq j$ istället för $xs[i] \neq xs[j]$:**

- Villkoret i uppgiften kräver att ett element inte ska multipliceras med sig självt på samma position i listan.
- Om en lista innehåller dubletter (t.ex. `xs = [2, 2, 3]`), skulle en kontroll av värdena (`xs[i] != xs[j]`) felaktigt filtrera bort multiplikationen mellan det första 2:an (index 0) och det andra 2:an (index 1).
- Genom att jämföra indexen ($i \neq j$) garanterar vi att vi enbart hoppar över när elementet ställs mot sin exakta position, oavsett vilka tal som finns i listan.

- **Hur man löser problemet:**

- Använd `range(len(xs))` för att iterera över index i och j istället för direkt över värdena.
- Lägg till villkoret `if i != j` i predikatdelen sist i listomfattningen.

- **Kodexempel:**

```
xs = [2, 3, 5, 7]

# Nästlad listomfattning med index och villkor i != j
ys = [xs[i] * xs[j] for i in range(len(xs)) for j in range(len(xs)) if i != j]

print(ys)
# Utskrift: [6, 10, 14, 6, 15, 21, 10, 15, 35, 14, 21, 35]
```

3. Fibonacci-generator och yield (Kapitel 6, Uppgift 6)

- **Viktigt att veta (Hur yield och tillstånd fungerar):**

- Till skillnad från `return`, som avslutar funktionen helt och raderar alla lokala variabler ur minnet, pausar `yield` funktionens exekvering och sparar hela dess tillstånd (variabelvärden och var i koden man befinner sig).
- Kodens före loopens exekvering utförs enbart en gång vid starten av generatoren för att initiera startvärdena.
- När nästa element begärs (t.ex. i en `for`-loop) återupptas exekveringen direkt på raden efter `yield`.

- **Hur man löser problemet:**

- Sätt upp startvärdena $a = 0$ och $b = 1$ innan loopens start.
- Använd en `while`-loop med villkoret $a \leq stop$.
- Generera värdet a med `yield a`, och uppdatera därefter båda variablerna parallellt: $a, b = b, a + b$.

- **Kodexempel:**

```
def fibonacci(stop):
    a, b = 0, 1
    while a <= stop:
        yield a
        a, b = b, a + b

# Exempel på användning:
for num in fibonacci(15):
    print(num, end=" ")
# Utskrift: 0 1 1 2 3 5 8 13
```

4. Generatoruttryck vs Generatorfunktioner (Kapitel 6, Uppgift 7 & 8)

- **Viktigt att veta (Skillnader och val av metod):**

- **Resultat:** Båda metoderna resulterar i ett generatorobjekt som utvärderas lat (beräknar ett element i taget när det efterfrågas).
- **Generatoruttryck:** Används med parenteser (`uttryck for item in iterabel if villkor`). Lämpar sig bäst när transformationen eller filtreringen är enkel och får plats på en enda rad.
- **Generatorfunktion:** Använder `def` och nyckelordet `yield`. Krävs när logiken kräver flera steg, sammanhangshantering (som `with open`), eller bearbetning av indata före utmatning.

- **Hur man löser Uppgift 7 (`map_gen`):**

- Eftersom uppgiften enbart kräver att en funktion appliceras på varje element kan funktionen returnera ett generatoruttryck direkt på en rad.

- **Kodexempel Uppgift 7:**

```
def map_gen(function, iterable):
    return (function(item) for item in iterable)

gen = map_gen(int, '123456')
print(sum(gen)) # Utskrift: 21
```

- **Hur man löser Uppgift 8 (`palindrome_rows`):**

- Eftersom filen måste öppnas och stängas säkert, samt varje rad behöver rensas från nyrads-tecken (`\n`), är en generatorfunktion med `yield` det säkraste och mest läsbara valet.

- **Kodexempel Uppgift 8:**

```
def is_palindrome(s):
    cleaned = s.strip()
    return cleaned == cleaned[::-1]

def palindrome_rows(filename):
```

```
with open(filename, 'r') as file:
    for line in file:
        yield is_palindrome(line)
```